

Einführung in die überwachte binäre Klassifizierung durch maschinelles Lernen

Thomas Daniel Ulbricht

Datum: 12.05.2026

Inhaltsverzeichnis

1. Abstract.....	1
2. Einführung.....	2
2.1 Die Vorteile des maschinellen Lernens in unserer Gesellschaft.....	2
2.2 Einführung in die Lernparadigmen des maschinellen Lernens.....	3
3. Klassische Regressionsmodelle.....	5
3.1 Das lineare Regressionsmodell.....	5
3.2 Das logistische Regressionsmodell.....	10
4. Das Learning Rate Scheduling als eine möglichkeit für ein effizienteres training	13
5. Künstliches neuronales Netz.....	16
5.1. Definition von künstlichen Neuronen.....	16
5.2. Das künstliche neuronale Netz.....	17
7. Literaturverzeichnis.....	18

Abstract

Diese Arbeit soll schrittweise in die überwachte binäre Klassifikation einführen. Zu Beginn werden die Vorteile des maschinellen Lernens für die Gesellschaft dargestellt. Nachfolgend wird erläutert, was maschinelles Lernen ist und in welchen Formen es in der Praxis Verwendung findet.

Anschließend wird das lineare Regressionsmodell erklärt und, darauf aufbauend, das logistische Regressionsmodell. Dann folgt eine Thematisierung des Learning Rate Scheduling. Zum Ende hin wird konzeptionell erläutert, was ein künstliches Neuron ausmacht und wie daraus ein künstliches neuronales Netzwerk entsteht.

Die Vorteile des maschinellen Lernens in unserer Gesellschaft

Maschinelles Lernen (ML, engl. Machine Learning) ist ein Teilbereich der künstlichen Intelligenz (KI). Es ermöglicht, durch das Trainieren eines Algorithmus mit bekannten Daten, diesem beizubringen schwer kategorisierbare und komplexe Aufgaben zu automatisieren, welche für die klassische Programmierung zu komplex oder aufwändig sind (SAP o. J.a). Das gibt dem Themenfeld eine große Präsenz im modernen Alltag und hat dadurch Vorteile, welche schwer wegzudenken sind. Im folgenden werden passende Beispiele genannt. Dazu gehört die Kommunikation. Durch ML Modelle, die speziell darauf trainiert sind, zu übersetzen, kann der Informationsaustausch stark beschleunigt werden (DeepL, 2025). Ein weiteres Beispiel ist, dass maschinelles Lernen das Potenzial, unser Leben in Zukunft durch KI-gestützte Sensoren im Auto (DGUV Forum, 2025) und durch eine bessere medizinische Versorgung sicherer zu gestalten. So sagen Takita und sein Team (2025) in einer Metaanalyse, in der die diagnostische Leistung von künstlicher Intelligenz mit der von Experten verglichen wird, dass: „[...]AI demonstrates promising diagnostic capabilities [...] Although it has not yet achieved expert level reliability, these findings suggest Potenzial for enhancing healthcare delivery and medical education when implemented with appropriate understanding of its limitations.“ (Takita et al., 2025, S. 1). Ein aktuelles Beispiel für ein Vorteil, den wir schon heute in unserer Gesellschaft nutzen können, lässt sich im Themenfeld der Psychologie finden. So zeigt sich in einer weiteren Metaanalyse, dass: „Conversational artificial intelligence (AI), particularly AI-based conversational agents (CAs), is gaining traction in mental health care. [...] The meta-analysis revealed that AI-based CAs significantly reduce symptoms of depression (Hedge's g 0.64 [95% CI 0.17–1.12]) and distress (Hedge's g 0.7 [95% CI 0.18–1.22]). [...]“ (Li et al., 2023, S. 1). Daraus lässt sich schließen, dass maschinelles Lernen bereits heute einen großen Mehrwert für unsere Gesellschaft bietet und zudem ein erhebliches Potenzial für die Zukunft bereithält, sofern dabei wichtige Grenzen berücksichtigt werden.

Einführung in die Lernparadigmen des maschinellen Lernens

Maschinelles Lernen ist ein Fachbereich, der Algorithmen untersucht, die durch Daten trainiert werden. Der Algorithmus lernt, verschiedene Aufgaben zu bewältigen, bei denen die klassische Programmierung zu aufwändig wäre (SAP, o. J.b). Dabei gibt es verschiedene Wege, das Modell lernen zu lassen. Der Großteil der Methoden lässt sich in drei Paradigmen kategorisieren: überwachtes Lernen, unüberwachtes Lernen und bestärkendes Lernen. Dabei ändern sich die Art der Daten, die Verarbeitung und die Praktikabilität (SimpleKI, o. J.).

Beim überwachten Lernen nutzt der Algorithmus ausgewählte Trainingsdaten, um zu lernen. Dabei sind die Daten gelabelt. Sie bestehen aus einem äußeren Einfluss und einem dazugehörigen, erwarteten, korrekten Ergebnis. Das Modell nutzt nun die Daten, um Vorhersagen zu treffen, die möglichst nahe am korrekten Ergebnis liegen sollten. Dieser Vorgang wird Forward Propagation genannt. Anfangs sind die Vorhersagen zufällig, da die Parameter, welche für die Bewertung zuständig sind, es ebenfalls sind (GeeksforGeeks, 2025.a). Das Programm sucht dann wiederholt nach den Gründen für die Fehler und korrigiert diese langsam durch die Änderung der Parameter. Dieser Vorgang wird Backpropagation genannt (Wikipedia, o. J.). Wenn das Modell in der Lage ist, überwiegend korrekte Ergebnisse zu erbringen, endet typischerweise die Backpropagation. Nun ist es möglich, dass das Modell, ungesehene Datensätze, im besten Fall korrekt bewerten zu lassen. Dann ist es generalisiert (Lokad, o. J.). Anwendung findet diese Methode beispielsweise in der Bilderkennung, Textklassifizierung (Alexander Thamm GmbH, 2022) und in Bereichen, in denen Vorhersagen benötigt werden, wie beim Wetter oder im Finanzwesen (Tobias Gantner, 2023). Zusammengefasst lässt sich sagen, dass überwachtes Lernen bei Aufgaben wie Klassifizierung und Prognosen zum Einsatz kommt (Wikipedia, o. J.).

Eine andere Herangehensweise bietet das unüberwachte Lernen. Hier wird das Modell mit Einflüssen trainiert, die kein Ergebnis beigelegt haben.

Die Aufgabe des Modells besteht darin, Muster, Korrelationen oder Gruppierungen zu erkennen (Codefinity, 2023). Das ist besonders hilfreich, um in Daten versteckte Zusammenhänge zu identifizieren, (IBM, o. J.) weshalb es oft Anwendung in Berichten wie der Marktforschung findet, um das Kaufverhalten von NutzerInnen zu analysieren und zu gruppieren. Auch im Bankwesen wird es benutzt, um Anomalien im Kaufverhalten frühzeitig zu erkennen (Codefinity, 2023).

Abschließend gibt es noch ein wichtiges Paradigma: Bestärkendes Lernen.

Bei dieser Methode nimmt zuerst ein sogenannter Agent die Daten in der Umgebung wahr. Dann entscheidet er, ob und welche Aktion er ausführen soll. Je nach Aktion wird er belohnt oder bestraft, was zu einem Lernprozess führt. Das macht ihn extrem attraktiv für Szenarien, in denen in Echtzeit Entscheidungen getroffen werden müssen wie im Finanzwesen oder in selbstfahrenden Autos (Studyflix, o. J.).

Das lineare Regressionsmodell

Um ein komplexes künstliches neuronales Netzwerk zu verstehen, ist es nützlich, das Konzept eines linearen Regressionsmodells gesehen zu haben. Lineare Regressionsmodelle haben sich die Eigenschaften der linearen Regressionsanalyse zunutze gemacht. Sie zählt zu den am häufigsten genutzten Analysemethoden in der Statistik und im maschinellen Lernen. Sie wird dazu genutzt, aus linearen Zusammenhängen Vorhersagen zu treffen (Marini Systems, o. J.). Die mathematische Formel lautet:

$$\hat{y} = b + w_1 \cdot x_1$$

$\hat{y}$ beschreibt die Vorhersage der Funktion, welche sich aus dem Bias (b), dem Gewicht (w_1) und dem Eingabewert (x_1) ergibt.

Sollte es mehrere Eingabewerte geben, werden diese mit einem separaten Gewicht multipliziert und mit der vorherigen Eingabe addiert. Diese modifizierte Funktion lässt sich wie folgt beschreiben:

$$\hat{y} = b + w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

(Google Developers, o. J.a).

Um das Modell zu trainieren, muss es zuerst Vorhersagen treffen, welche anschließend korrigiert werden.

Der erste Schritt, um das zu tun, ist die Forward Propagation.

Die Eingabewerte werden in die Funktion eingesetzt, welche als Ergebnis die Vorhersage liefert (kiberatung.de, o. J.).

Anhand der Vorhersagen gilt es dann, den Error (Fehler) zu berechnen. Die Formel dazu lautet:

$$E = y - \hat{y}$$

y steht hierbei für das Label (Tatsächliches Ergebnis), welches subtrahiert mit der Vorhersage den Error E wiedergibt (Wikipedia contributors, o. J.).

Mit dem Error wird anschließend der Loss berechnet, welcher den Fehler des Modells für einen einzelnen Datenpunkt in einem numerischen Wert festhält. Dabei gibt es viele unterschiedliche Ansätze, in denen jede Variante andere Interessensfelder abdeckt.

Anschließend folgt die Berechnung der Kosten durch die Kostenfunktion. Sie liefert einen numerischen Wert für einen durchschnittlichen Loss über alle Trainingsdaten. Dadurch unterscheiden sich die Formel-Definitionen nur geringfügig von den Loss-Definitionen, da nur der Loss-Wert im Nachhinein gemittelt wird (EnjoyAlgorithms, o. J.). Das führt dazu, dass es genauso so viel Loss- wie Kostenfunktionen gibt.

Im Folgenden ist eine Tabelle zu sehen, die einige der populären Kostentypen und ihre Funktionsdefinition zeigt.

Kostentypen	Definition
MAE(Mean Absolute Error)	$C = \frac{1}{N} \sum_{i=1}^N E_i $
MSE(Mean Squared Error)	$C = \frac{1}{N} \sum_{i=1}^N (E_i)^2$
RMSE(Root Mean Squared Error)	$C = \sqrt{\frac{1}{N} \sum_{i=1}^N (E_i)^2}$

C stellt die Kostenfunktion dar. N steht für die Anzahl der Datenpunkte. i ist der Index. E ist der vorher ausgerechnete Error. Die Tabelle basiert auf (Google Developers, o. J.b).

Nun ist es wichtig, für das richtige Ziel die passende Methode auszuwählen, um das bestmögliche Ergebnis zu erzielen. Grob zusammengefasst ist es am vorteilhaftesten, die Wahl danach zu treffen, wie stark Noise (ausreißende Daten) in den endgültigen Wert einfließt, wie interpretierbar dieser Wert ist und wie gut er differenzierbar bleibt.

Beispielsweise geht MAE durch das Nicht-Quadrieren des Errors weniger auf Noise ein und lässt zusätzlich eine leichtere Interpretation des Wertes zu, was ihn einfach und intuitiv macht. Aufgrund dessen wird MAE häufig genutzt, wenn es um Regressionsaufgaben geht. Ein Problem jedoch ist, dass sich die Formel nicht bei 0 differenzieren lässt, was sie eher unattraktiv für viele Optimierungsalgorithmen macht, wie zum Beispiel Gradient Descent.

MSE hingegen bestraft Noise stärker durch das Quadrieren des Errors, was es sensibler macht gegenüber ausreißenden Datenpunkten. Jedoch leidet die Interpretierbarkeit des Wertes darunter. Durch die Differenzierbarkeit eignet es sich sehr gut für einen Optimierungsalgorithmus wie den Gradient Descent. Diese Eigenschaften sorgen dafür, dass es einer der meistgenutzten Loss-Typen für das maschinelle Lernen ist (Medium, 2024).

Für den folgenden Kontext ist es wichtig, das Konzept der Lernrate zu verstehen. Die Lernrate ist ein Hyperparameter, also ein Wert, der nicht dynamisch mit den Datenpunkten interagiert. Deshalb wird er vor dem Training festgelegt (DataBraineo, o. J.). So definiert die Lernrate wie stark Parameter, wie beispielsweise der Bias und das Gewicht, nach jedem Optimierungs-Durchlauf angepasst werden (IBM, o. J.c). Um die richtige Lernrate zu finden, bedarf es einer Balance zwischen Geschwindigkeit und Genauigkeit. Ist die Lernrate zu klein eingestellt, braucht das Modell zu lange, um Fehler zu korrigieren. Wenn die Lernrate zu hoch eingestellt ist, kann es passieren, dass der im Folgenden diskutierte Optimierungsalgorithmus um die für den Kontext beste Lösung oszilliert (Everpure, o.J.).

Nun folgt die Backward Propagation of Error (Backpropagation). Diese wird nur im Kontext des maschinellen Lernens angewendet. Sie ermöglicht es, mithilfe eines Gradientenabstiegs-Algorithmus, iterativ den Bias und die Gewichte beziehungsweise das Gewicht anzupassen, so dass die Kostenfunktion ein lokales Minimum erreicht und infolge der Kostenwert so gering wie möglich ist (IBM, o. J.b). Im besten Fall stellt sich der Punkt als globales Minimum

heraus, was bei dem linearen Regression Modell garantiert ist. Jedoch ist es nicht das genaue Ziel der Backpropagation.

Der am häufigsten verwendete Optimierungsalgorithmus im maschinellen Lernen, also auch für neuronale Netze, ist der Vanilla Gradient Descent (GD). Übersetzt ins Deutsche bedeutet das einfacher Gradientenabstieg (Khanfer, 2021). Da es mittlerweile viele Abwandlungen davon gibt, kennzeichnet das Wort Vanilla im Namen, dass es sich um die Grundform handelt (Stack Exchange, o. J.). Der Algorithmus leitet die Kostenfunktion zum jeweiligen Parameter ab und multipliziert diesen dann mit der Lernrate. Dann wird dieser vom Parameter subtrahiert. Dieser Vorgang wiederholt sich, bis der Bias und das Gewicht so angepasst sind, dass das Modell für den Kontext zufriedenstellend vorhersagen kann. Die Formeln dazu sehen wie folgt aus:

$$w_i := w_i - \eta \cdot \frac{\delta C}{\delta w_i}$$
$$b := b - \eta \cdot \frac{\delta C}{\delta b}$$

η steht hier für die Lernrate und C für die Kostenfunktion (GeeksforGeeks, o.J.b).

In der folgenden Abbildung 1 ist ein Code zu sehen, welcher die oben erklärten Konzepte umsetzt.

```
trainingdata:      list[int] = [10,100,1,5]      # Die Trainingsdaten <-- Quadratmeter eines Hauses.
labels_for_training_data:list[int] = [500,5000,50,250] # Die Labels sind über den Array-Index verbunden
und geben den Preis des Hauses wieder

weight:float = 0.00001 # Auf eine Zufallszahl gesetzt.
bias:float = 0.0002     # Auf eine Zufallszahl gesetzt.

def linear_regression_function(func_input:int)-> float:
    return bias + weight * func_input

def error_function(labels:int,prediction:float)-> float:
    return labels - prediction

def loss_function(error:float)-> float:
    return error**2          #L2

def cost_function(loss)-> float:
    return 1/2 * loss        #MSE

epochs:int = 10000
learning_rate:float = 0.0000001

for epoch in range(epochs):
    for i in range(len(trainingdata)):
        #forward_propagation
        prediction = linear_regression_function(trainingdata[i])

        error:float = error_function(labels_for_training_data[i],prediction)
        loss: float = loss_function(error)
        cost: float = cost_function(loss)

        # Backpropagation of Error mit dem Vanilla Gradient Descent (GD)

        weight += learning_rate * error * trainingdata[i] #Die Ableitung der Kostenfunktion nach w
        #Die Ableitung der Kostenfunktion nach w
        #Die Ableitung der Kostenfunktion nach w
        bias    += learning_rate * error                    #Die Ableitung der Kostenfunktion nach b
        #Die Ableitung der Kostenfunktion nach b
        #Die Ableitung der Kostenfunktion nach b

#model test
print(linear_regression_function(1)) # output: 50.562943147888085 <-- Die Vorhersage des Modells, wie
teuer ein Haus mit einem Quadratmeter wäre.
print(linear_regression_function(9)) # output: 450.4946509104428  <-- Die Vorhersage des Modells, wie
teuer ein Haus mit neun Quadratmeter wäre.
print(linear_regression_function(10)) # output: 500.4861143807621  <-- Die Vorhersage des Modells, wie
teuer ein Haus mit zehn Quadratmeter wäre.
```

Abbildung 1

Das binäre logistische Regressionsmodell

Nun folgt das logistische Regressionsmodell. Im Gegensatz zum linearen Regressionsmodell wird es nicht benutzt, um Vorhersagen zu berechnen, sondern eine Wahrscheinlichkeit. Diese zeigt, welche der zwei zur Verfügung stehenden Kategorien am wahrscheinlichsten ist (Amazon Web Services, o. J.).

Es kann beispielsweise verwendet werden, um E-Mails zu analysieren und sie anschließend in die Kategorien Spam oder Nicht-Spam zu klassifizieren. Wird eine E-Mail zu 50% der Kategorie Spam zugeordnet, heißt es, dass das Modell unsicher ist. Liegt die Wahrscheinlichkeit bei 99%, ist sich das Modell sehr sicher, dass es sich um Spam handelt (Levine, 2025).

Der erste zentrale Unterschied im Aufbau eines logistischen Regressionsmodells im Vergleich zu einem linearen Regressionsmodell liegt in der Anwendung der Sigmoid-Funktion auf der Vorhersage der linearen Funktion.

Die Vorhersage einer linearen Funktion wird in diesem Kontext oft auch Log-Odds genannt und mit z abgekürzt (Google Developers, o. J.c).

Der Funktionswert der Sigmoid-Funktion kann als Wahrscheinlichkeit interpretiert werden. Sie zeigt, wie sicher das Modell ist, dass der Input zu der entsprechenden Kategorie gehört. Die Formel wird wie folgt definiert:

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

e steht hier für die Eulersche Zahl (Wikipedia, 2025).

Die Sigmoid-Funktion ordnet die Vorhersage in einen Bereich zwischen 0 und 1 ein. Dadurch entstehen zwei Kategorien. Werte nahe der 0 zählen beispielsweise eher zu der Kategorie "Nicht-Spam". Werte nahe der 1 zählen eher zu der Kategorie Spam. Um das zu bewerkstelligen, ist die Funktion S-förmig, so wie es in Abbildung 2 sichtbar ist.

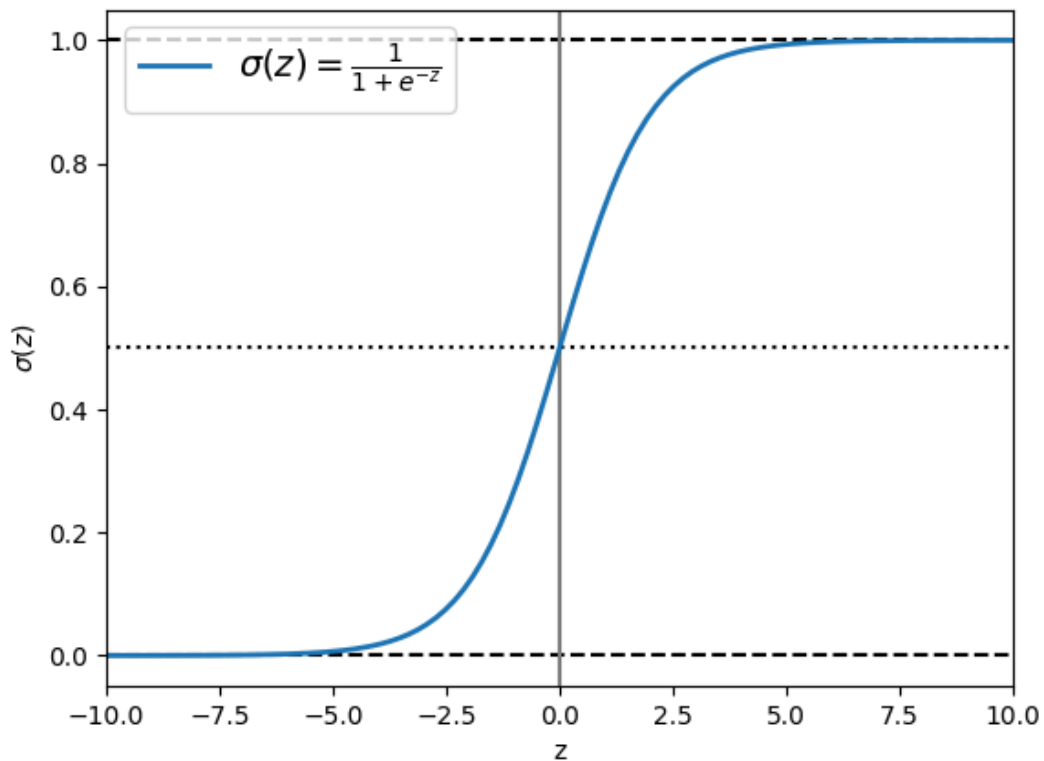


Abbildung 2 basiert auf (Google Developers, o. J.c).

Ein weiterer Unterschied liegt in der Loss- und Kostenfunktion. Hier wird beispielsweise statt der MSE Function der gemittelte Binary Cross Entropy Loss verwendet. Der Grund dafür ist, dass durch die Klassifizierung in zwei Kategorien die Ausgabe des Models bestenfalls nahe an der 0 oder der 1 liegt, nicht dazwischen. Daher ist MSE eher unpassend. Es bestraft das Modell nur schrittweise nach unten. Der gemittelte Binary Cross Entropy Loss hingegen bestraft selbstbewusste, aber falsche Zuordnungen hart, um sie schnell auf die andere Seite des Spektrums zu ziehen (Savle, 2023). Die Formel für den gemittelten Binary Cross Entropy Loss sieht wie folgt aus:

$$\text{Gemittelter Binary Cross Entropy Loss} = -\frac{1}{N} \sum_{i=1}^N y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$$

N steht für die Anzahl der gelabelten Beispiele im Dataset. i ist der Index. y ist das Label. $\hat{y}$ ist die Vorhersage (Google Developers, 2025).

Um eine bessere Vorstellung zu haben, inwiefern gerade der Logarithmus gut ist, falsche Aussagen hart zu bestrafen, hilft die Abbildung 3.

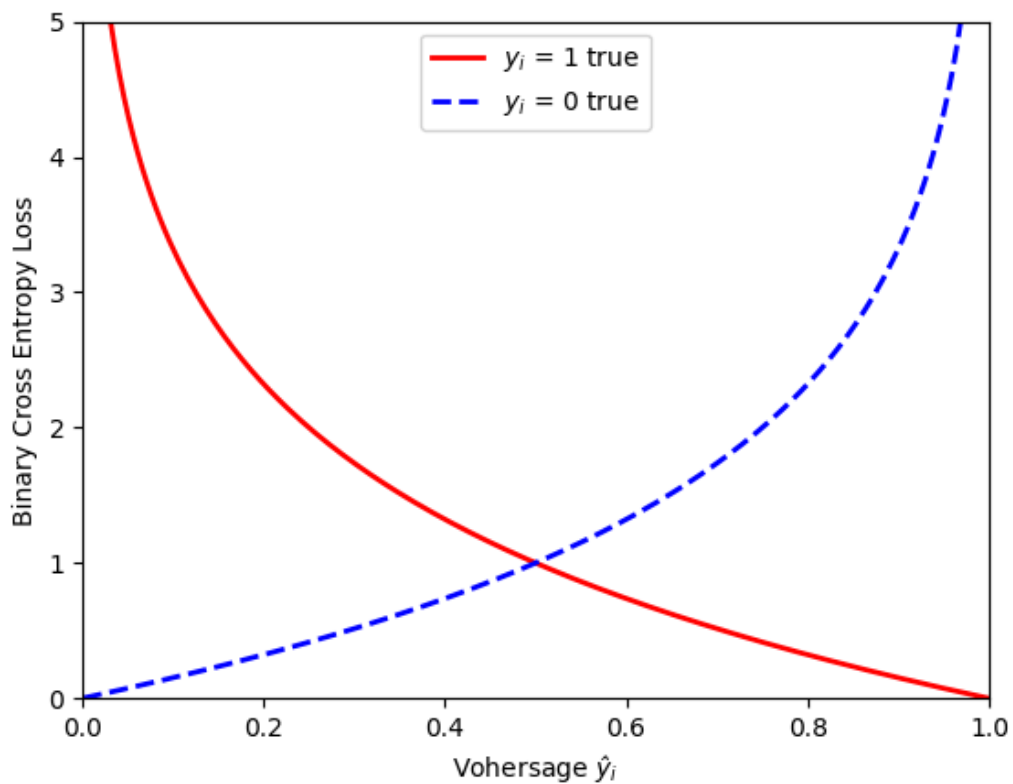


Abbildung 3 basiert auf (Daniel Ramos et al., 2018).

Es ist sichtbar, dass, sollte die Vorhersage $\hat{y}_i$ des Modells weit vom Label y_i entfernt sein, der Binary Cross Entropy Loss einen hohen Funktionswert zurückgibt. Ist die Vorhersage $\hat{y}_i$ jedoch nahe am Label y_i , gibt sie einen niedrigen Funktionswert wieder.

Das Learning Rate Scheduling als eine Möglichkeit für ein effizienteres training

In der Praxis wird versucht, Algorithmen bestmöglich zu optimieren, um so viel Ressourcen wie möglich zu sparen. Ein attraktives Beispiel bietet das Learning Rate Scheduling.

Da die Lernrate dafür entscheidend ist, wie stark der Gradient in die Richtung des Minimums zeigt, hat sie einen großen Einfluss auf die Effizienz. Eine zu hohe Lernrate führt dazu, dass das Modell dauerhaft um das Minimum schwankt oder vom Minimum weg divergiert, so wie es links in der Abbildung 4 sichtbar ist. Sollte allerdings die Lernrate zu niedrig sein, dauert das Training oft zu lange (Martin, 2023). Erkenbar ist das rechts in der Abbildung 4.

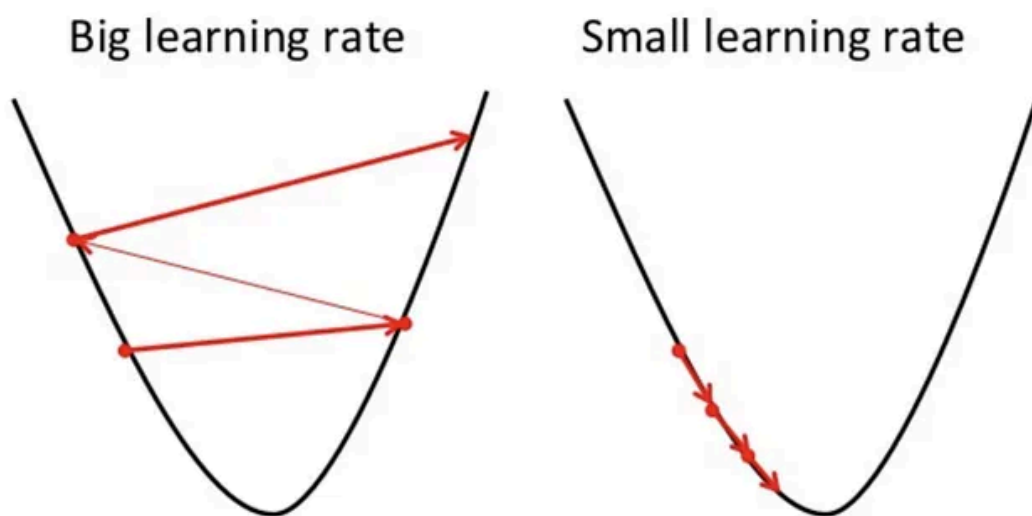


Abbildung 4 (Mishra, 2023)

Aufgrund dessen ist es ratsam, einen Mittelweg zu gehen, welcher im *Learning Rate Scheduling* liegt. Hier wird die Lernrate über das Training angepasst. Häufig wird sie am Anfang groß angesetzt und verringert sich über die Zeit immer weiter. So kann das Modell anfänglich, wenn der Optimalwert weit entfernt ist, sich schnell nähern und später langsam feinere Abstimmungen vornehmen.

Der optimale Verlauf ist in Abbildung 5 sichtbar (Martin, 2023).

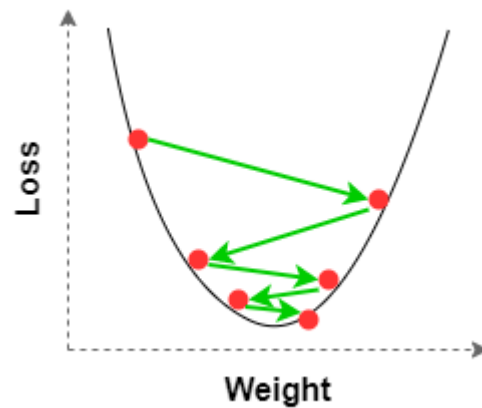


Abbildung 5 (Pramoditha, 2022)

Hier gibt es ähnlich wie bei der Kostenfunktion mehrere Ansätze. Eine der unkompliziertesten und oft benutzten Varianten ist der „Step Decay Learning Rate Schedule“ Die Formel lässt sich wie folgt definieren:

$$a_t = a_0 \cdot \gamma^{\lfloor t/S \rfloor}$$

t stellt die aktuelle Epoche (ein Durchgang durch den Datensatz) dar. a_0 ist die Lernrate am Anfang. S definiert, nach wie vielen Epochen die Lernrate sich verändert. γ ist die Stärke der Veränderung. Hierbei rundet $\lfloor t/S \rfloor$ auf die nächste kleinere ganze Zahl ab.

Abbildung 6 zeigt, wie sich die Größe der Lernrate in einem Szenario verhält, bei dem die Hyperparameter wie folgt definiert werden: $a_0 = 0.01$, $\gamma = 0,5$, $S = 15$.

Die y-Achse stellt die Lernrate dar, die x-Achse die Epochen.

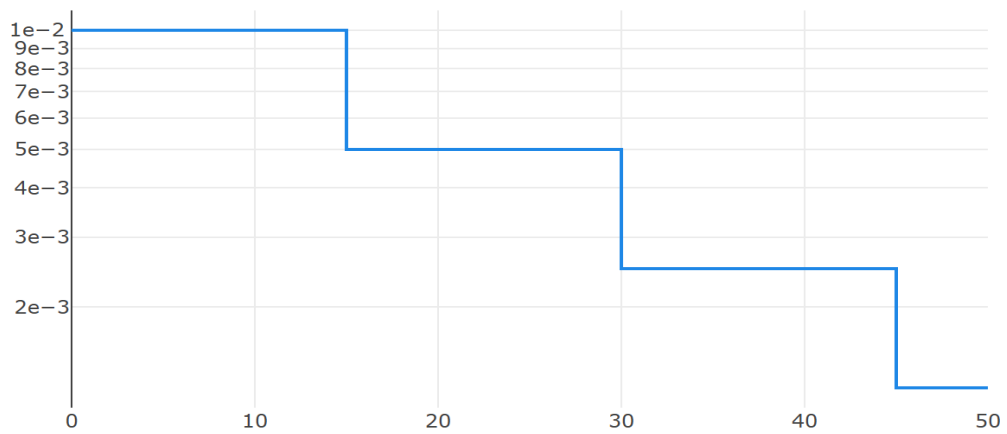


Abbildung 6

Es ist erkennbar, dass die Lernrate sich alle 15 Epochen halbiert
(ApX Machine Learning, o. J.).

Definition von künstlichen Neuronen

Ein künstliches Neuron bildet das Fundament für die meisten Machine-Learning-Modelle wie z.B. die schon aufgezeigte binäre logistische Regression. Künstliche Neuronen, wie sie in Abbildung 7 zu sehen sind, bestehen aus mehreren Komponenten.

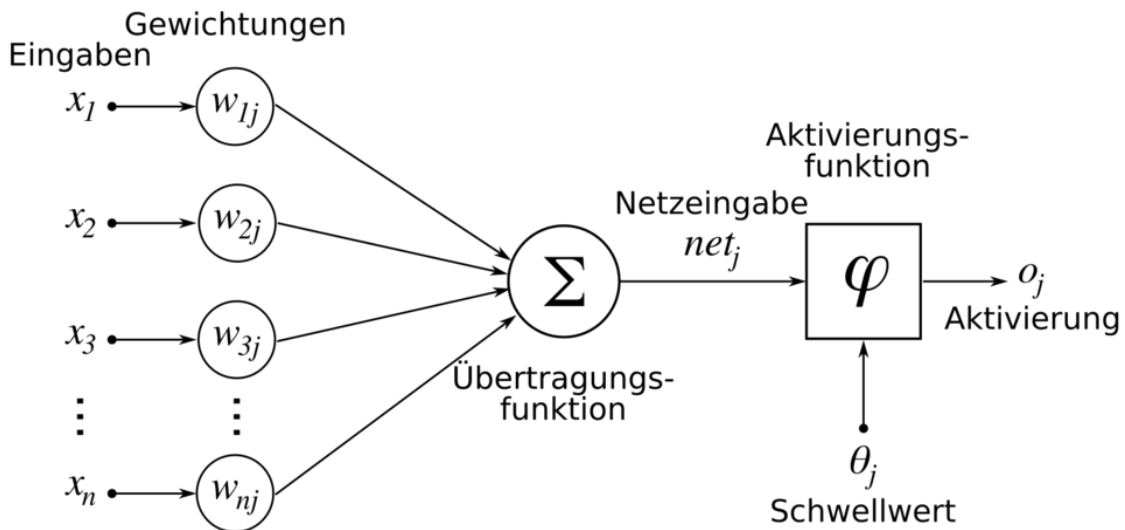


Abbildung 7

Die Eingabe kann von außen kommen oder von einem vorherigen Neuron. Der Input wird dann meist mit einem Gewicht multipliziert. Dieser Vorgang wird Linearkombination genannt. Anschließend folgt die Übertragungsfunktion. Hier werden alle gewichteten Eingaben multipliziert und danach beispielsweise durch einen Parameter modifiziert. Im zuvor beschriebenen binären logistischen Regressionsmodell wurde dafür der Bias verwendet, den Funktionswert nannte man dort Log-Odds. Allgemein wird der Funktionswert als Netzeingabe betitelt. Die Netzeingabe kommt anschließend in die Aktivierungsfunktion. Im vorherigen Modell stellte das die Sigmoid-Funktion dar, jedoch gibt es viele Funktionen, die dafür in Frage kommen. Sie bringen ihre eigenen Vor- und Nachteile mit sich. Überschreitet der Aktivierungsfunktionswert den Schwellenwert, übergibt das Neuron die Aktivierung. Ein Schwellenwert ist jedoch nicht zwingend notwendig (Wikipedia, 2026).

Das künstliche neuronale Netzwerk

Das binäre logistische Regressionsmodell ist in der Lage, einfache Datensätze bei einem guten Training richtig zu klassifizieren, jedoch kommt es bei komplexen Daten mit hochkomplexen Zusammenhängen schnell an seine Grenzen. Beispielsweise wäre ein logistisches Regressionsmodell nicht in der Lage, hochauflösende Bilder zu verarbeiten. Die Lösung liegt darin, das Modell an sich komplexer zu gestalten, durch das Hinzufügen von künstlichen Neuronen. Werden mehrere künstliche Neuronen miteinander verbunden, indem die Aktivierung als neuer Input für die nächsten Neuronen verwendet wird, führt das zu einem künstlichen neuronalen Netzwerk (KNN) (Wikipedia, 2026). Künstliche Neuronen sind meist in 3 verschiedene Kategorien von Schichten eingeteilt. In Abbildung 8 ist das schematisch dargestellt.

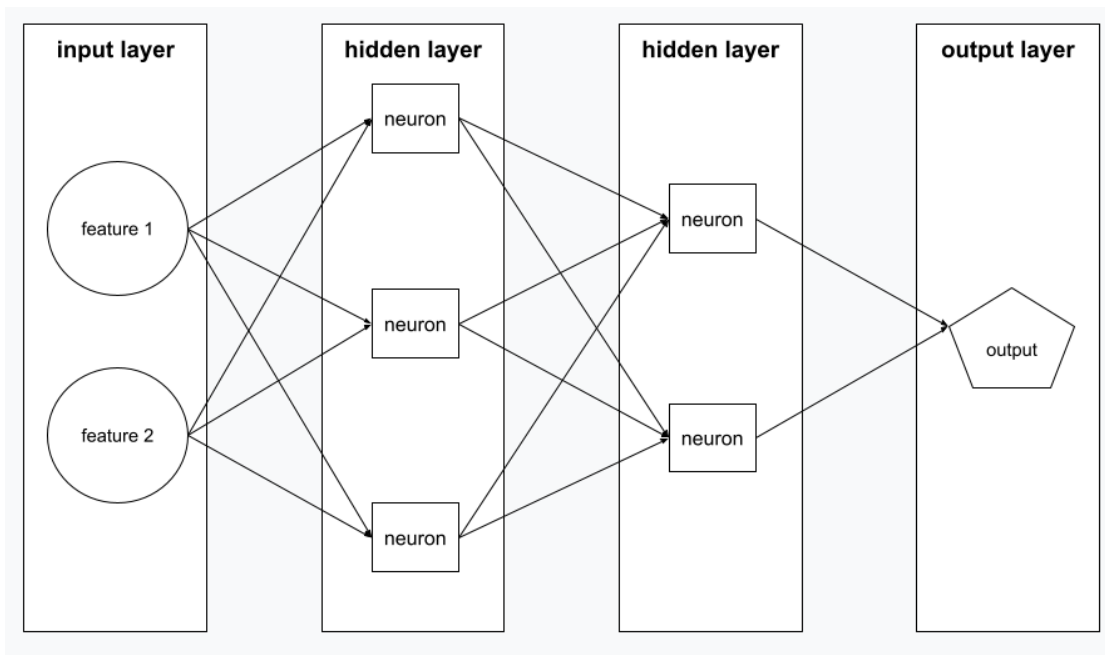


Abbildung 8 (Google Developers, o. J.d)

Durch die Input layer werden Informationen, wie zum Beispiel Bilder, in das neuronale Netz gebracht. Anschließend kommt die hidden layer, beziehungsweise die hidden layers, hier werden die Informationen erneut gewichtet. Die output layer berechnet die finale Antwort des Modells (Wuttke, 2024).

Literaturverzeichnis

DeepL. (2025). So revolutioniert KI die globale Kommunikation: Ergebnisse des Language AI Reports. DeepL Blog. Abgerufen am 12. Mai 2026, von <https://www.deepl.com/de/blog/language-ai-report-findings>

Takita, H., Kabata, D., Walston, S. L., Tatekawa, H., Saito, K., Tsujimoto, Y., Miki, Y., & Ueda, D. (2025). A systematic review and meta-analysis of diagnostic performance comparison between generative AI and physicians. Abgerufen am 12. Mai 2026, von <https://www.nature.com/articles/s41746-025-01543-z>

DGUV Forum. (2025). KI im Straßenverkehr: Vorteile und Herausforderungen für die Vision Zero. Ausgabe 3/2025. Abgerufen am 12. Mai 2026, von <https://forum.dguv.de/ausgabe-3-2025/ki-im-strassenverkehr-vorteile-und-herausforderungen-fuer-die-vision-zero/>

Li, H., Zhang, R., Lee, Y.-C., Kraut, R. E., & Mohr, D. C. (2023). Systematic review and meta analysis of AI-based conversational agents for promoting mental health and well-being. npj Digital Medicine, 6, 236. Abgerufen am 12. Mai 2026, von <https://doi.org/10.1038/s41746-023-00979-5>

SAP. (o. J.b). Was ist Machine Learning? Abgerufen am 25. November 2025, von <https://www.sap.com/germany/products/artificial-intelligence/what-is-machine-learning.html>

SimpleKI. (o. J.). Die 3 Arten des maschinellen Lernens. Abgerufen am 25. November 2025, von <https://simpleki.de/arten-machine-learning/>

GeeksforGeeks. (2025a). What is forward propagation in neural networks. Abgerufen am 25. November 2025, von <https://www.geeksforgeeks.org/deep-learning/what-is-forwardpropagation-in-neural-networks/>

Wikipedia. (o. J.). Backpropagation. In Wikipedia.

Abgerufen am 25. November 2025, von

<https://de.wikipedia.org/wiki/Backpropagation>

Lokad. (o. J.). Generalisierung (Prognostizieren). Abgerufen

am 25. November 2025, von

<https://www.lokad.com/de/generalisierung/>

Alexander Thamm GmbH. (2022, 10. August). Supervised Learning: Kompakt erklärt. Abgerufen am 26. November 2025, von

<https://www.alexanderthamm.com/de/blog/supervisedmachine-learning/>

Wikipedia. (o. J.). Generalisierung (Prognostizieren). Abgerufen am 25. November 2025, von https://de.wikipedia.org/wiki/Überwachtes_Lernen

Tobias Gantner. (2023, August). Supervised learning. Abgerufen am 26. November 2025, von <https://tobias-gantner.com/ki-robotik/supervised-learning/>

Codefinity. (2023, Dezember). Supervised vs Unsupervised Learning. Abgerufen am 26. November 2025, von

<https://codefinity.com/blog/Supervised-vs-Unsupervised-Learning?>

IBM. (o. J.a). What is unsupervised learning? Abgerufen am 26. November 2025, von <https://www.ibm.com/think/topics/unsupervised-learning>

Studyflix. (o. J.). Reinforcement Learning. Abgerufen am 26. November 2025, von <https://studyflix.de/informatik/reinforcement-learning-8509>

Marini Systems. (o. J.). Lineare Regression. Abgerufen am 9. Dezember 2025 von <https://marini.systems/de/glossar/lineare-regression/>

Google Developers. (o. J.a). Lineare Regression – Machine Learning Crash-Course. Abgerufen am 9. Dezember 2025 von

<https://developers.google.com/machine-learning/crashcourse/linear-regression?hl=de>

kiberatung.de. (o. J.). Forward Propagation (Vorwärtspropagierung). Abgerufen am 12. Januar 2026 von https://www.kiberatung.de/ki-glossar/forward-propagationvorwartspropagierung?utm_medium=organic&utm_source=kiberatung.de%2Fki-glossar%2Fforward-propagation-vorwartspropagierung&utm_content=google

Wikipedia. (o. J.). Errors and residuals. In Wikipedia. Abgerufen am 4. Januar 2026 von https://en.wikipedia.org/wiki/Errors_and_residuals

EnjoyAlgorithms. (o. J.). Loss and Cost Function in Machine Learning. Abgerufen am 12. Januar 2026 von <https://www.enjoyalgorithms.com/blog/loss-and-cost-functions-in-machinelearning>

Google Developers. (o. J.b). Loss (Verlustfunktion) – Linear Regression | Machine Learning Crash Course. Abgerufen am 16. Dezember 2025 von <https://developers.google.com/machine-learning/crash-course/linear-regression/loss>

Medium. (2024, 13. Juni). Comprehensive Guide to Machine Learning Loss Functions and Evaluation Metrics Explained: PART 1 (Regression). Abgerufen am 16. Dezember 2025 von https://medium.com/%40rakeshbobbati_85122/comprehensive-guide-to-machine-learning-loss-functions-and-evaluation-metrics-explained-part-1-3e16e495c549

DataBraineo. (o. J.). Hyperparameter. Abgerufen am 12. Januar 2026 von <https://databraineo.com/data-science/hyperparameter/>

IBM. (o. J.c). Learning Rate: Bedeutung und Rolle im maschinellen Lernen. Abgerufen am 12. Januar 2026 von <https://www.ibm.com/de-de/think/topics/learning-rate>

Everpure. (o.J.) What Is Learning Rate in Machine Learning?
Abgerufen am 23. April 2026 von
<https://www.purestorage.com/knowledge/what-is-learning-rate.html>

IBM. (o. J.b). Backpropagation: Was es ist und wie es neuronale Netze trainiert.
Abgerufen am 12. Januar 2026 von <https://www.ibm.com/de/de/think/topics/backpropagation>

Khanafer, A. H. (2021, 16. August). Deep Learning Part 2: Vanilla vs Stochastic Gradient Descent. Abgerufen am 12. Januar 2026 von
<https://medium.com/geekculture/deep-learningpart-2-vanilla-vs-stochastic-gradient-descent-6bcecc26fd51>

Stack Exchange. (o. J.). *What does “vanilla” mean?* Abgerufen am 23. februar 2026 von
<https://stats.stackexchange.com/questions/295180/what-does-vanilla-mean>

GeeksforGeeks. (o. J.b). Gradient Descent in Linear Regression. Abgerufen am 12. Januar 2026 von
<https://www.geeksforgeeks.org/machine-learning/gradient-descent-in-linearregression/>

Amazon Web Services (AWS). (o.J.). *The Difference Between Linear Regression and Logistic Regression*. Abgerufen am 10. Februar 2026 von
<https://aws.amazon.com/de/compare/the-difference-between-linear-regression-and-logistic-regression/>

Miles Levine(2025,12, Mai)Can We Predict Spam? Using Logistic Regression to Filter Email Noise. Aufgerufen am 2. April 2026 von
<https://medium.com/inst414-data-science-tech/can-we-predict-spam-using-logistic-regression-to-filter-email-noise-ee6d45cc8330>

Wikipedia. (2025). *Logistische Regression*. Abgerufen am 11. Februar 2026 von https://de.wikipedia.org/w/index.php?title=Logistische_Regression&oldid=2617283
54

Google Developers. (o. J.c). *Logistische Regression: Berechnung einer Wahrscheinlichkeit mit der Sigmoidfunktion*. Abgerufen am 11. Februar 2026 von <https://developers.google.com/machine-learning/crash-course/logistic-regression/sigmoid-function?hl=de>

Google Developers. (2025). *Logistic Regression: Loss and Regularization*. Abgerufen am 12. Februar 2026 von <https://developers.google.com/machine-learning/crash-course/logistic-regression/loss-regularization?hl=de>

Harshad Savle(2023, 9. Sep) why do we use LOG-LOSS as an evaluation metric instead of MSE? Abgerufen am 5. April 2026 von <https://harshad-savle.medium.com/why-do-we-use-log-loss-as-an-evaluation-metric-instead-of-mse-ea95a5da8226>

Daniel Ramos, Javier Franco-Pedroso, Alicia Lozano-Diez, & Joaquin Gonzalez-Rodriguez(2018 ,20. Mar)Deconstructing Cross-Entropy for Probabilistic Binary Classifiers
Abgerufen am 6. April 2026 von <https://pubmed.ncbi.nlm.nih.gov/33265299/>

Martin, T. (2023, 9. Juli). *A (Very Short) Visual Introduction to Learning Rate Schedulers (With Code)*. Abgerufen am 23. februar 2026 von <https://medium.com/@theom/a-very-short-visual-introduction-to-learning-rate-schedulers-with-code-189eddfdb00>

Mishra, M. (2023, 28. Mai). *The learning rate: A hyperparameter that matters*. Abgerufen am 23. februar 2026 von <https://mohitmishra786687.medium.com/the-learning-rate-a-hyperparameter-that-matters-b2f3b68324ab>

Pramoditha, R. (2022, 21. September). How to Choose the Optimal Learning Rate for Neural Networks. Abgerufen am 23. Februar 2026, von <https://towardsdatascience.com/how-to-choose-the-optimal-learning-rate-for-neural-networks-362111c5c783/>

ApX Machine Learning. (o. J.). Step Decay Schedules. Abgerufen am 1. März 2026, von <https://apxml.com/courses/deep-learning-regularization-optimization/chapter-7-optimization-refinements-tuning/step-decay-lr-schedule>

geeksforgeeks. (2025, 10. Dezember). Underfitting and Overfitting in ML. Abgerufen am 1. März 2026, von <https://www.geeksforgeeks.org/machine-learning/underfitting-and-overfitting-in-machine-learning/>

Wikipedia (2026, 11. April) Künstliches Neuron. Abgerufen am 27. April 2026 von [https://de.wikipedia.org/wiki/Künstliches_Neuron](https://de.wikipedia.org/wiki/K%C3%BCnstliches_Neuron)

Laurenz Wuttke (2024, 1. Februar) Künstliche Neuronale Netzwerke: Definition, Einführung, Arten und Funktion. Abgerufen am 27. April 2026 von <https://datasolut.com/neuronale-netzwerke-einfuehrung/>

Google Developers. (o. J. d) Glossar zum maschinellen Lernen: ML-Grundlagen. Abgerufen am 27. April 2026 von <https://developers.google.com/machine-learning/glossary/fundamentals?hl=de>